

A Comparison of Voronoi Fracturing in Vulkan and OpenGL

By Samuel Lloyd

Context

As destruction effects in games have grown in usage, complexity, and scale, the need for performance has grown in tandem

A commonly used algorithm for destruction is Voronoi fracturing, where a mesh is sliced against a generated Voronoi diagram.

Voronoi is a well research algorithms, that has already been well optimised. However, there is a constant demand for performance in applications such as game engines where Voronoi is commonly implemented.

Context

There is a lack of research on the graphical API that best suites Voronoi, such as OpenGL or Vulkan.

Graphics APIs can differ in the performance they offer, due to differing levels of control, and what GPU architecture they were designed for.

As a result of this, choosing the right graphical API can offer significant performance gains.

Justification

Optimising Voronoi for the correct Graphical API will offer clearer insight into the correct API to use when building an application that involves Voronoi.

This could benefit those with lower end hardware, as it could potentially lower the hardware barrier of applications that utilise Voronoi if they are more performant, and benefit the education sector when teaching about which API has more modern relevance.

Question and Hypothesis

- From this, the research question can be concluded as:
How does OpenGL compare to vulkan in performance for a 3D Voronoi fracturing of a complex mesh?
- And from this, we can derive the hypotheses

Hypothesis	Null Hypothesis
OpenGL will perform slower than Vulkan at Voronoi cell generation on modern GPU's	OpenGL will be the same speed as Vulkan at Voronoi cell generation on Modern GPU's
OpenGL will perform slower than Vulkan at Voronoi clipping on modern GPU's	OpenGL will be the same speed as Vulkan at Voronoi clipping on Modern GPU's

Method

- An application for running a test of a Voronoi fracture on a 3D mesh will be created.
- This will utilise a bridge interface between the two API's, so that the fracture code is the same on both API's, reducing bias.

Method

- To solve my research question, a test on a set of 10, 100, and 1000 points will be repeated 145 times, as G*Power calculated that as the needed sample size.
- The time will be recorded after each fracture. After all sets have been recorded, they will be used to create a box-plot for each set using python's pandas.
- A Wilcoxon-Mann Whitney test will be performed on the data, for each point set of 10, 100, and 1000
- This will give me clear results for how each API performs.

Challenges

- Implementing a bridge interface for both OpenGL and Vulkan is a complex task.
- Voronoi diagrams and clipping in 3D is a known complex problem with few resources online.

Lit results

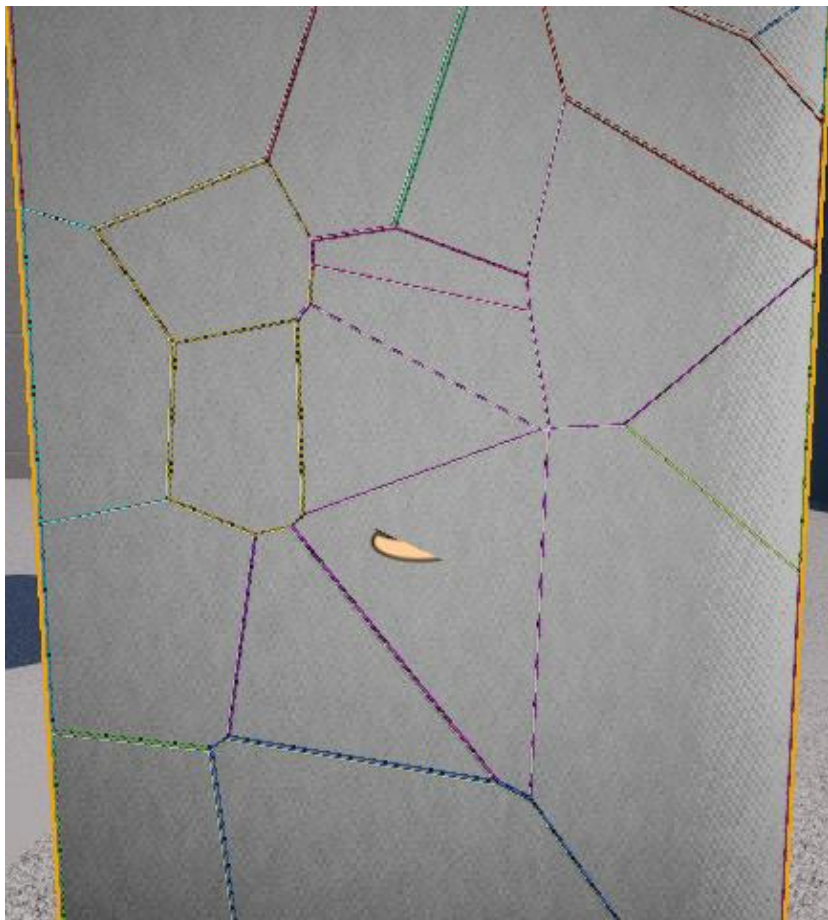
Vulkan will perform better than OpenGL on modern GPUs, but OpenGL could perform better on older GPUs.

Constrained Delaunay triangulation will be used due to its efficiency $O(n^2)$ and its ability to retain the shape of a polygon on triangulation, unlike Delaunay.

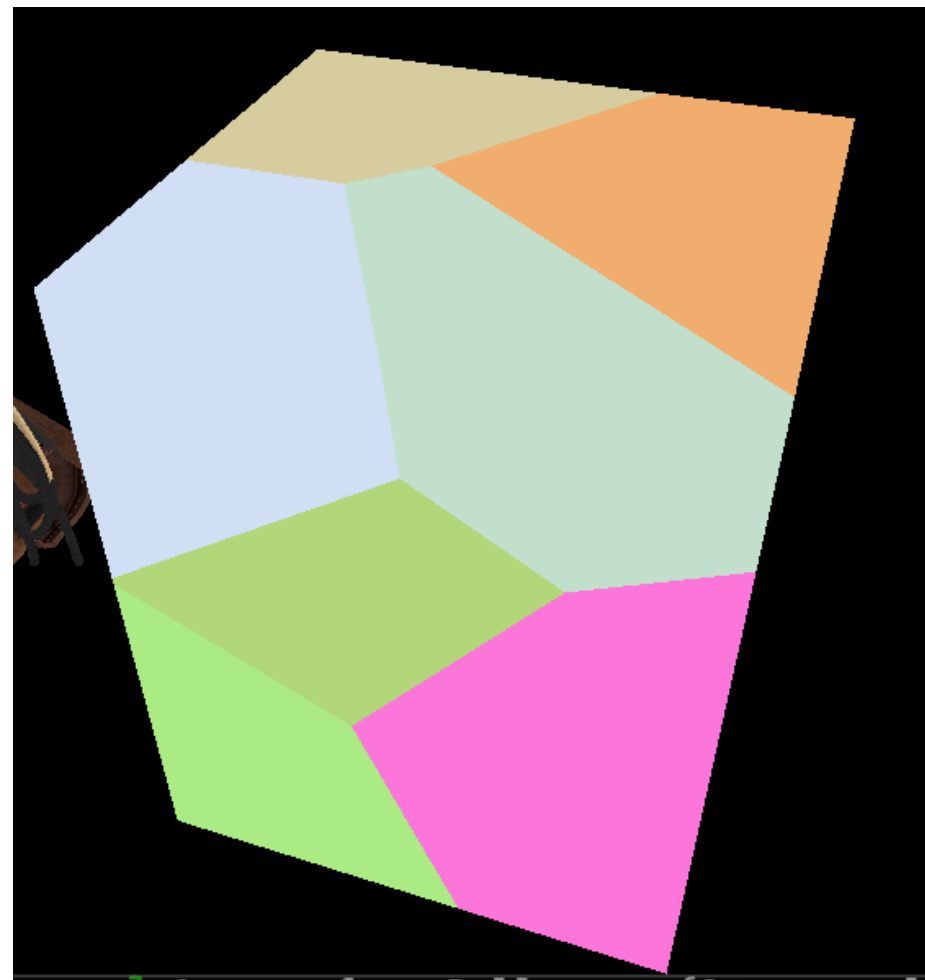
Poisson sampling will be used as it is optimised, and will ensure a set distance between points, which is important as points being too close could result in floating point errors.

There isn't any research that explicitly compares Vulkan and OpenGL in Voronoi.

State of the art



Unreal Engine



My Implementation

Quality Assurance

- Polarity testing using unit test will be performed, helping to test that the program is functioning, and how well it responds to unexpected inputs.
- Standard Readability measurement tests such as lines of code, maintainability index and cyclomatic complexity will be conducted to assess the readability of the program.
- Assert statements will be used where appropriate to improve maintainability.

Legal

- Doesn't violate GDPR.
- Voronoi diagrams and Voronoi clipping are in the public domain.
- Vulkan and OpenGL are in the public domain, however the implementations are not. However, as we don't modify the drivers, this is not an issue.

Social

- Could lower the hardware barrier for applications that use Voronoi. This would increase accessibility.
- Could shift the development trends if a significant performance advantage is shown by one of the API. This could result in increased hiring for that API, and a change in the educational priority's.

Ethical

- This project has been approved as low risk.
- Does not violate GDPR, Nuremburg code, Bcs code of conduct.

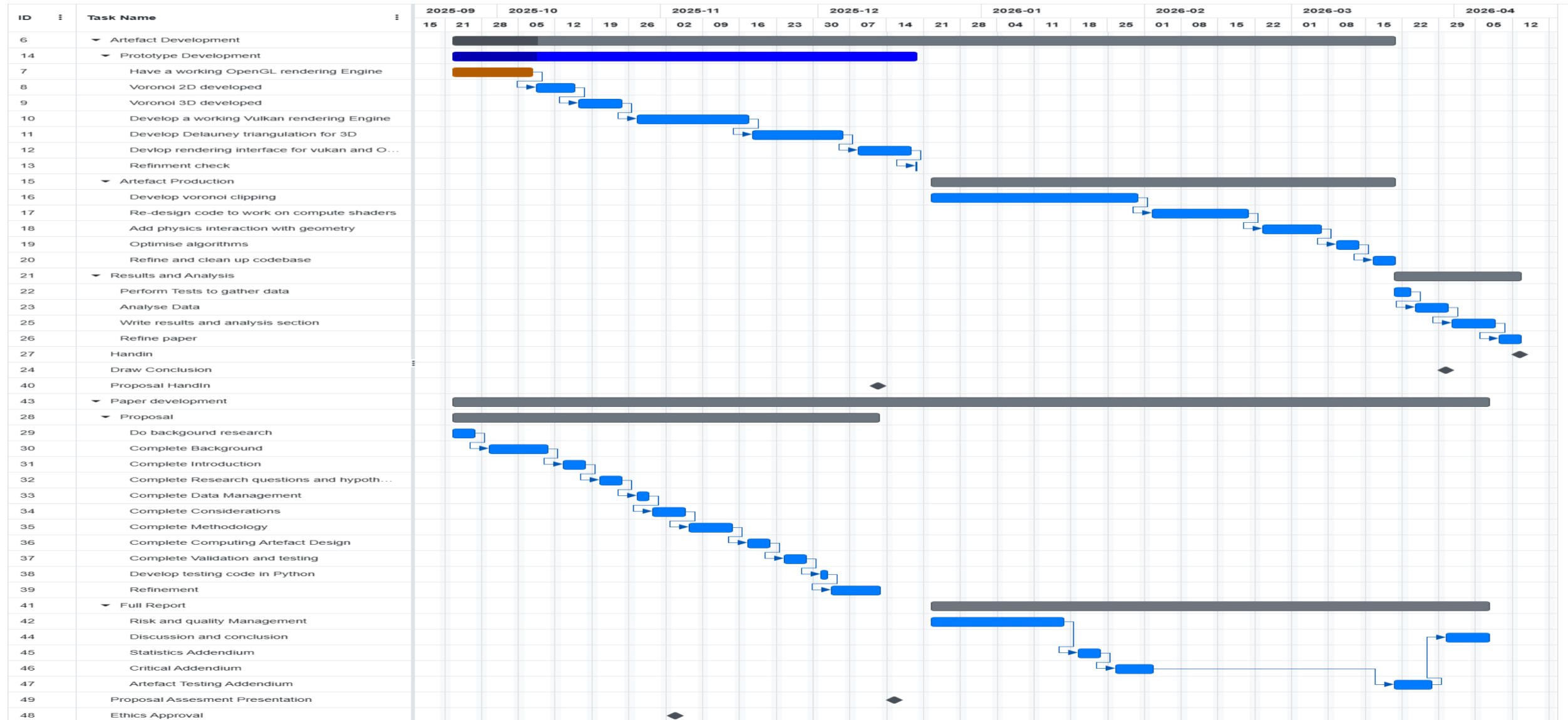
Professional

- Adheres to the bcs code of conduct of not claiming a level of competence one does not possess by stating results have not been tested on a wide variety of machines.
- Follows Nuremburg code by providing research which will only benefit society, which this paper does by conducting research to clarify the more optimised API for Voronoi destruction.

Sustainability

- Games as a whole consume a large amount of electricity, therefore optimising a widely used destruction algorithm could lead to a decrease in the overall consumption of electricity in games that use Voronoi destruction.

Project plan



Thank you for listening!